Московский авиационный институт
(национальный исследовательский университет)

Факультет информационных технологий и прикладной математики

Кафедра вычислительной математики и программирования

Лабораторная работа №1 по курсу «Информационный поиск»

Студент: А. М. Голев
Преподаватель: А. А. Кухтичев
Группа: М8О-401Б
Дата:
Оценка:
Подпись:

Москва, 2026

Лабораторная работа №1 «Добыча корпуса документов»

Необходимо подготовить корпус документов, который будет использован при выполнении остальных лабораторных работ:

- Скачать его к себе на компьютер. В отчёте нужно указать источник данных.
- Ознакомиться с ним, изучить его характеристики. Из чего состоит текст? Есть ли дополнительная мета-информация? Если разметка текста, какая она?
- Разбить на документы.
- Выделить текст.
- Найти существующие поисковики, которые уже можно использовать для поиска по выбранному набору документов (встроенный поиск Википедии, поиск Google с использованием ограничений на URL или на сайт). Если такого поиска найти невозможно, то использовать корпус для выполнения лабораторных работ нельзя!
- Привести несколько примеров запросов к существующим поисковикам, указать недостатки в полученной поисковой выдаче.

В результатах работы должна быть указаны статистическая информация о корпусе:

- Размер «сырых» данных.
- Количество документов.
- Размер текста, выделенного из «сырых» данных.
- Средний размер документа, средний объём текста в документе.

1 Описание

1 Источники данных

Для построения корпуса использовались два источника данных.

Подкорпус текстов песен. Список популярных треков получается из Last.fm API (метод `chart.gettoptracks`), а текст песни для каждого трека — из бесплатного Lyrics.ovh API (запрос вида `/v1/{артист}/{название}`). Last.fm используется с бесплатным ключом API, Lyrics.ovh ключа не требует.

Подкорпус информации о записях. Список популярных артистов получается из Last.fm API (метод `chart.gettopartists`), затем для каждого артиста из MusicBrainz API загружается список записей (поиск `/ws/2/recording`) и подробная информация о каждой записи: длительность, жанры, идентификатор MBID.

2 Характеристики корпуса

Корпус состоит из двух подкорпусов. Каждый документ — HTML-страница, сгенерированная программой, плюс отдельный JSON-файл с метаданными.

В подкорпусе текстов песен документ содержит название трека, исполнителя и текст песни (блок `div.lyrics`). Метаданные: артист, название, длина текста, число слов, время загрузки, источник данных.

В подкорпусе записей MusicBrainz документ содержит название записи, исполнителя, длительность, жанры и MBID. Метаданные: MBID, артист, название, время загрузки.

Разметка документов — HTML, метаданные хранятся в формате JSON.

3 Разбиение на документы

Корпус разбит на документы по принципу «один объект — один документ»: каждый трек сохраняется в файл `track_{N}.html` с метаданными `track_{N}_full.json`, каждая запись MusicBrainz — в файл `recording_{mbid}.html` с метаданными `recording_{N}.json`. Такое разбиение позволяет обрабатывать документы независимо друг от друга.

4 Выделение текста

Сырые данные — HTML-страницы. Текст выделяется из размеченных блоков: для текстов песен — содержимое блока `div.lyrics`, для записей — текстовое содержимое информационных блоков (название, исполнитель, длительность, жанры). HTML-теги, CSS-оформление и служебные элементы (стили, футер, статистика) отбрасываются.

5 Существующие поисковики

Для поиска по выбранному набору документов уже можно использовать:

- **Google с ограничением по сайтам** — для поиска текстов песен: тексты тех же песен доступны на публичных сайтах (genius.com, azlyrics.com), поэтому запросы вида `site:genius.com {артист} {название}` находят те же документы, что и в корпусе;
- **встроенный поиск MusicBrainz** (search.musicbrainz.org) — для поиска записей по артисту и названию.

6 Примеры запросов и недостатки выдачи

1. Google: `Olivia Rodrigo "stupid song"site:genius.com` — текст находится, но его версия может отличаться от скачанной с Lyrics.ovh (другая пунктуация и форматирование).
2. Google: `Taylor Swift "Patient Zero"текст песни` — в выдаче много дублей одного и того же текста на разных сайтах, страницы с рекламой и новости, а не только сам текст.
3. Поиск MusicBrainz: `artist:"Radiohead"recording:"Just"` — нужная запись находится, но вместе с ней десятки похожих (ремастеры, live-версии, сборники), из которых сложно выбрать нужную.

Общие недостатки: поисковики не ограничены нашим корпусом (выдают документы, которых в нём нет), а поиск MusicBrainz работает только по метаданным, а не по содержимому документа.

7 Статистика по корпусу

Статистика приведена для целевого размера корпуса — 30000 документов (по 15000 в каждом подкорпусе) документам.

Показатель	Тексты песен	Записи	Всего
Размер «сырых» данных	~57 МБ	~41 МБ	~98 МБ
Количество документов	15000	15000	30000
Размер выделенного текста	~20 МБ	~6 МБ	~26 МБ
Средний размер документа	~3,8 КБ	~2,7 КБ	~3,3 КБ
Средний объём текста в документе	~1,3 КБ	~0,4 КБ	~0,9 КБ

2 Исходный код

Программа `parser.py` написана на Python 3 и использует библиотеку `requests`. Она состоит из двух классов: `LyricsOvhParser` (подкорпус текстов песен) и `MusicBrainzParser` (подкорпус записей).

Этапы работы `LyricsOvhParser`:

1. получение страницы популярных треков из Last.fm API;
2. для каждого трека — запрос текста из Lyrics.ovh;
3. сохранение HTML-документа и JSON-метаданных.

Этапы работы `MusicBrainzParser`:

1. получение страницы популярных артистов из Last.fm API;
2. поиск записей артиста в MusicBrainz (постранично, по 100 записей);
3. загрузка подробной информации о каждой записи;
4. сохранение HTML-документа и JSON-метаданных.

При написании кода пришлось учесть особенности реальных API: временные сбои серверов (повторные попытки запроса), зависание отдельных запросов (таймауты), ограничение частоты запросов MusicBrainz (пауза 1,2 с между запросами) и отсутствие части текстов (около 15–20% треков возвращают 404, такие треки пропускаются).

Получение списка популярных треков из Last.fm (с повторными попытками при сбое):

```
def get_popular_tracks(self, page=1, page_size=20):
    api_key = "..."
    url = "https://ws.audioscrobbler.com/2.0/"
    params = {
        "method": "chart.gettoptracks",
        "api_key": api_key,
        "format": "json",
        "page": page,
        "limit": page_size
    }
    for attempt in range(3):
        try:
            response = self.session.get(url, params=params, timeout=30)
            if response.status_code == 200:
                data = response.json()
```

```
                tracks = data.get("tracks", {}).get("track", [])
                track_list = []
                for track in tracks:
                    artist = track.get("artist", {}).get("name", "")
                    title = track.get("name", "")
                    if artist and title:
                        track_list.append((artist, title))
                return track_list
            except Exception:
                pass
            time.sleep(2 * (attempt + 1))
        return None
```

Запрос текста песни из Lyrics.ovh (таймаут и повтор при ошибке сервера):

```
def get_lyrics(self, artist, title):
    url = f"{self.base_url}/{quote_plus(artist)}/{quote_plus(title)}"
    for attempt in range(2):
        try:
            response = self.session.get(url, timeout=30)
            if response.status_code == 200:
                lyrics = response.json().get("lyrics", "")
                if lyrics and len(lyrics.strip()) > 50:
                    return lyrics.strip()
                return None
            elif response.status_code == 404:
                return None
            elif response.status_code >= 500 and attempt == 0:
                continue
            return None
        except Exception:
            if attempt == 0:
                continue
            return None
    return None
```

Основной цикл загрузки текстов (различаются ошибка API и конец списка, защита от вечной пагинации при отказе сервиса):

```
while downloaded < max_tracks:
    tracks = self.get_popular_tracks(page, page_size)
    if tracks is None: # API error -> retry the page
        page_errors += 1
        if page_errors >= 3:
            break
        continue
    if not tracks: # track list is exhausted
        break
    for artist, title in tracks:
        lyrics = self.get_lyrics(artist, title)
        if lyrics:
```

```
            self.save_lyrics_html(artist, title, lyrics, downloaded + 1)
            downloaded += 1
        else:
            consecutive_failures += 1
            if consecutive_failures >= 50:
                break
    page += 1
```

Основной цикл загрузки записей MusicBrainz (топ-артисты Last.fm, постраничный поиск записей, пропуск повторов по MBID):

```
while downloaded < max_recordings:
    artists = self.get_top_artists(artist_page, 50)
    if not artists:
        break
    for artist in artists:
        offset = 0
        while downloaded < max_recordings:
            recordings = self.search_recordings_by_artist_api(
                artist, limit=100, offset=offset)
            if recordings is None: # search failed -> skip artist
                break
            if not recordings: # no more recordings
                break
            for recording in recordings:
                if recording["id"] in processed_ids:
                    continue
                processed_ids.add(recording["id"])
                data = self.get_recording_details(recording["id"])
                if data:
                    self.save_recording_html(data, recording["id"])
                    downloaded += 1
                time.sleep(1.2) # MusicBrainz rate limit: 1 req/s
            if len(recordings) < 100:
                break
            offset += 100
    artist_page += 1
```

3 Выводы

Выполнив первую лабораторную работу по курсу «Информационный поиск», я научился собирать корпус документов из публичных API. Основное, что я узнал нового:

- работа с реальными API (Last.fm, Lyrics.ovh, MusicBrainz): ключи API, параметры запросов, разбор ответов в формате JSON;
- обработка сбоев сети при загрузке большого объёма данных: таймауты, повторные попытки, ограничение частоты запросов — без этого загрузка корпуса обрывается на первых же ошибках (что и произошло в первой версии программы);
- разбиение данных на документы и хранение метаданных отдельно от содержимого документа;
- подсчёт статистики корпуса: размер сырых данных, количество документов, размер выделенного текста, средние размеры.

Полученный корпус будет использован в следующих лабораторных работах для построения поискового индекса и реализации поиска. Работа с существующими поисковиками (Google с ограничением по сайтам, встроенный поиск MusicBrainz) показала их недостатки — дубликаты, нерелевантную выдачу и поиск только по метаданным; именно эти недостатки предстоит устранить собственным поисковиком.

Список литературы

[1] Маннинг, Рагхаван, Шютце *Введение в информационный поиск* — Издательский дом «Вильямс», 2011. Перевод с английского: доктор физ.-мат. наук Д. А. Клюшина — 528 с. (ISBN 978-5-8459-1623-4 (рус.))

[2] Список использованных источников оформлять нужно по ГОСТ Р 7.05-2008

[3] Last.fm API Documentation // https://www.last.fm/api

[4] Lyrics.ovh API // https://api.lyrics.ovh

[5] MusicBrainz API Documentation // https://musicbrainz.org/doc/MusicBrainz_API

[6] Requests: HTTP for Humans // https://docs.python-requests.org